The ACL mask

The ACL mask defines the maximum permissions that can be generated for named users, the group owner and named groups. It does not restrict the permissions of the file owner or other users. All files and directories that implement ACLs will have an ACL mask.

The mask can be viewed with *getfacl* and can be explicitly set with *setfacl*. It will be calculated and added automatically if it is not explicitly set, but it could also be inherited from a parent directory's default mask setting. By default, the mask is recalculated whenever any of the affected ACLs are added, modified or deleted.

ACL permission precedence

When determining whether a process (a running program) can access a file, file permissions and ACLs are applied as follows:

- If a process is running as the user that owns the file, then the file's user ACL permissions apply.
- If the process is running as a user that is listed in a named user ACL entry, then the named user ACL permissions apply (as long as it is permitted by the mask).
- If the process is running as a group that matches the group owner of the file or as a group with an explicit name group ACL entry, then the matching ACL permissions apply (as long as it is permitted by the mask).
- Otherwise, the file's other ACL permissions apply.

Securing files with ACLs

Changing ACL file permissions: Use *setfacl* to add, modify or remove standard ACLs on files and directories. ACLs use the normal file system representation *r* for read permission, *w* for write permission, and *x* for execute permission. A '**-**' (dash) indicates that the relevant permission is absent. When (recursively) setting ACLs, an upper-case *X* can be used to indicate that *execute* permissions should only be set on directories and not regular files, unless the file already has the relevant *execute* permissions. This is the same behaviour as *chmod*.

Adding or modifying an ACL: ACLs can be set via the command line using *–m* or passed via a file using *–M* (use '**-**' (dash) instead of a file name for *stdin*). These two are the modify options; they add new ACL entries or replace specific existing ACL entries on a file or directory. Any other existing ACL entries on the file or directory remain untouched.

Note: Use the *--set* or *--set-file* options to completely replace the ACL settings on a file. When first defining an ACL of a file, if the *add* operation does not include setings for the file owner, group-owner or other permissions, then they will be set based on the current standard file permissions (these are also known as the base ACLs and cannot be deleted), and a new *mask* value will be calculated and added as well.

To add or modify a user or named user ACL, use the following command:

```
# setfacl -m u:name:rX file
```

If a name is left blank, then it applies to the file owner; otherwise, the name can be a user name or UID value. In this example, the permissions granted are *read-only* and if already set, *execute* (unless the file was a directory, in which case the directory gets the *execute* permissions set to allow directory search).

ACL file owner and standard file owner permissions are equivalent; consequently, using *chmod* on the file owner permissions is equivalent to using *setfacl* on them. *chmod* has no effect on named users.

To add or modify a group or named group ACL, use the following command:

```
# setfacl -m g:name:rw file
```

This follows the same pattern for adding or modifying a user ACL. If the name is left blank, then it applies to the group owner. Otherwise, specify a group name or GID value for a named group. The permissions are *read* and *write* in this example. *chmod* has no effect on any group permissions for files with ACL settings, but it updates the ACL mask.

To add or modify the other ACL, use the following command:

```
#setfacl -m o::- file
```

Other only accepts permission settings. It is common for permissions to be set to '**-**' (dash), which specifies that other users have no permissions, but any of the standard permissions can be specified.

ACL *other* and standard *other* permissions are equivalent, so using *chmod* on the *other* permissions is equivalent to using *setfacl* on them. Add multiple entries using the same command, and comma-separate each of the entries, as follows:

```
# setfacl -m u::rwx, g:sodor:rX,o::- file
```

This will set the file owner to *read*, *write* and *execute*; set the named group *sodor* to read-only and conditional execute, and restrict all other users to no permissions. The group owner will maintain the existing file or ACL permissions and other named entries will remain unchanged.

Using *getfacl* as input

The output from *getfacl* can be used as input to *setfacl*:

```
# getfacl file-A | setfacl --set-file=- file-B
```